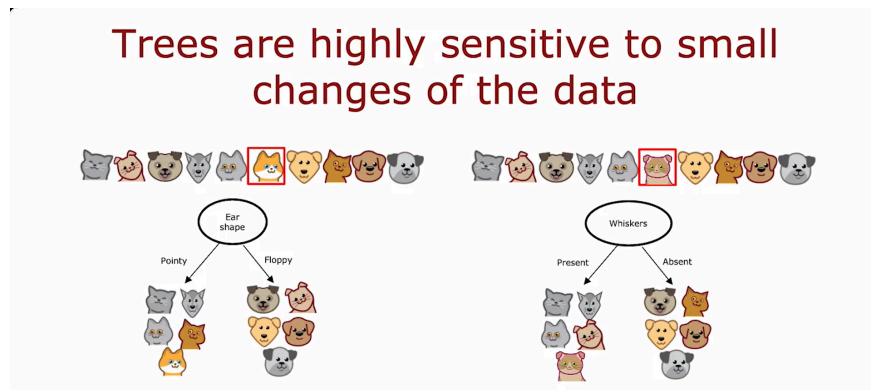


Tree ensembles

Using multiple decision trees

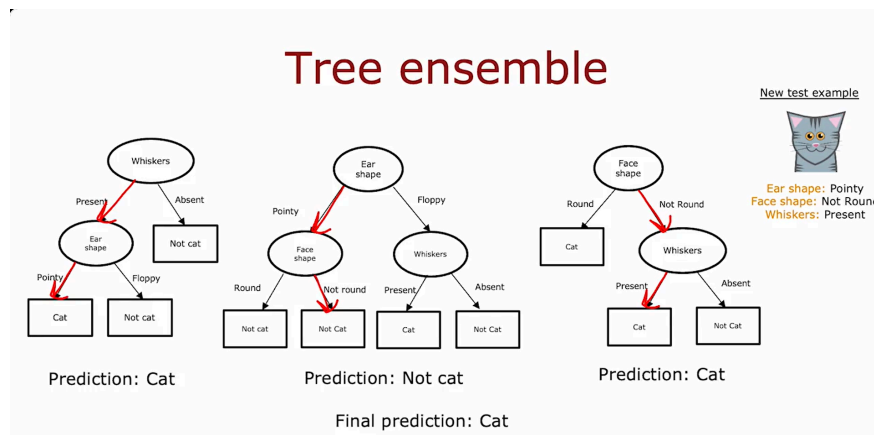


With the example that we've been using, the best feature to split on at the root node turned out to be the ear shape resulting in these two subsets of the data and then building further sub trees on these two subsets of the data.

But it turns out that if you were to take just one of the ten examples and change it to a different cat so that instead of having pointy ears, round face, whiskers absent, this new cat has floppy ears, round face, whiskers present, with just changing a single training example, the highest information gain feature to split on becomes the whiskers feature instead of the ear shape feature.

As a result of that, the subsets of data you get in the left and right sub-trees become totally different and as you continue to run the decision tree learning algorithm recursively, you build out totally different sub trees on the left and right. The fact that changing just one training example causes the algorithm to come up with a different split at the root and therefore a totally different tree, that makes this algorithm just not that robust.

That's why when you're using decision trees, you often get a much better result, that is, you get more accurate predictions if you train not just a single decision tree but a whole bunch of different decision trees.



This is what we call a **tree ensemble**, which just means a collection of multiple **trees**. We'll see, in the next few videos, how to construct this ensemble of trees.

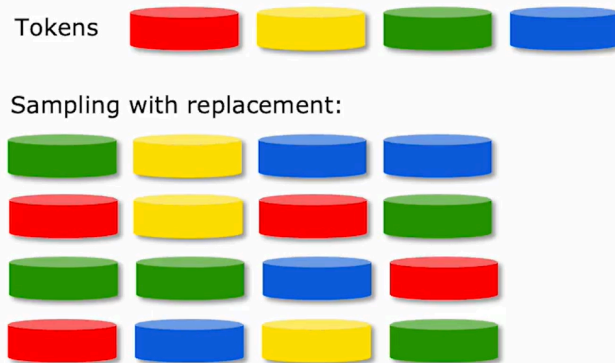
But if you had this ensemble three trees, each one of these is maybe a plausible way to classify cat versus not cat. If you had a new test example that you wanted to classify, then what you would do is run all three of these trees on your new example and get them to vote on whether it's the final prediction.

This test example has pointy ears, a not round face shape and whiskers are present and so the first tree would carry out inferences like this and predict that it is a cat. The second tree's inference would follow this path through the tree and therefore predict that is not cat. The third tree would follow this path (the red path) and therefore predict that it is a cat.

These three trees have made different predictions and so what we'll do is actually get them to vote. The majority votes of the predictions among these three trees is, cat. The final prediction of this ensemble of trees is that this is a cat which happens to be the correct prediction.

Sampling with replacement

Sampling with replacement



Put these 4 tokens into a black box, can't see what inside the box.

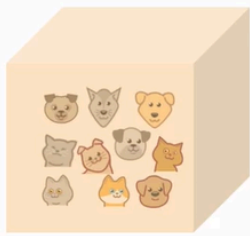
The first time, we randomly take 1 token in the box → the green token

Then put the green token back into the the box, shake it up.

The second time, take another token in the box → the yellow

If you were to repeat this sampling with replacement procedure multiple times, if you were to do it again, you might get red, yellow, red, green, or green, green, blue, red. Or you might also get red, blue, yellow, green. Notice that the with replacement part of this is critical because if I were not replacing a token every time I sample, then if I were to pour four tokens from my bag of four, I will always just get the same four tokens. That's why replacing a token after I pull it out each time, is important to make sure I don't just get the same four tokens every single time. The way that sampling with replacement applies to building an ensemble of trees is as follows.

Sampling with replacement



	Ear shape	Face shape	Whiskers	Cat
	Pointy	Round	Present	1
	Floppy	Not round	Absent	0
	Pointy	Round	Absent	1
	Pointy	Not round	Present	0
	Floppy	Not round	Absent	0
	Pointy	Round	Absent	1
	Pointy	Round	Present	1
	Floppy	Not round	Present	1
	Floppy	Round	Absent	0
	Pointy	Round	Absent	1

Let's say we get this training example. Then we put it back into the bag, and then again randomly pick out one training example and so you get that. You pick again and again and again. Notice now this fifth training example is identical to the second one that we had out there. But that's fine. You keep going and keep going, and we get another repeats the example, and so on and so forth.

Until eventually you end up with 10 training examples, some of which are repeats. You notice also that this training set does not contain all 10 of the original training examples, but that's okay. That is part of the sampling with replacement procedure.

Random forest algorithm

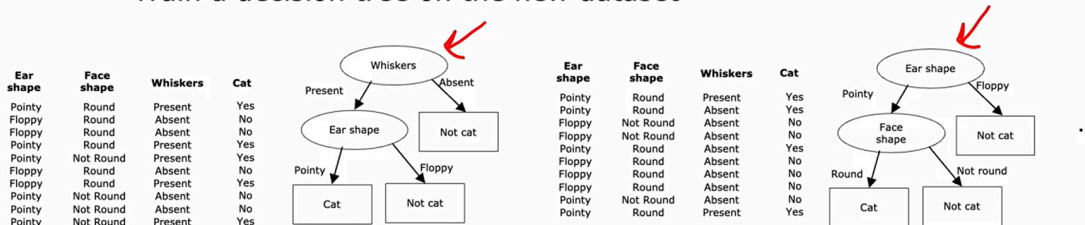
Generating a tree sample

Given training set of size m

For $b = 1$ to B :

Use sampling with replacement to create a new training set of size m

Train a decision tree on the new dataset



Bagged decision tree

Just to give this particular algorithm a name. This specific instance creation of tree ensemble is sometimes also called a bagged decision tree. And that refers to putting your training examples in that virtual bag. And that's why also we use the let us lower case "b" an uppercase "B" here because that stands for bag.

There's one modification to this algorithm that will actually make it work even much better and that changes this algorithm the bagged decision tree into the random forest algorithm. The key idea is that even with this sampling with replacement procedure sometimes you end up with always using the same split at the root node and very similar splits near the root note. That didn't happen in this particular example where a small change the trainings that resulted in a different split at the root note.

Randomizing the feature choice

At each node, when choosing a feature to use to split, if n features are available, pick a random subset of $k < n$ features and allow the algorithm to only choose from that subset of features.

$$k = \sqrt{n}$$

Random forest algorithm

if n is large like dozens or even hundreds we could use $k = \text{square root of } (n)$.

- **Cây quyết định (Decision Tree):** Mỗi cây quyết định là một mô hình học máy đơn giản, phân chia dữ liệu dựa trên các thuộc tính để đưa ra dự đoán. Tuy nhiên, một cây quyết định đơn lẻ có thể dễ bị overfitting (quá khớp với dữ liệu huấn luyện) và không ổn định.
- **Hợp lực (Ensemble):** Random Forest khắc phục nhược điểm của cây quyết định bằng cách kết hợp nhiều cây quyết định lại với nhau. Mỗi cây được huấn luyện trên một phần dữ liệu khác nhau và có thể có cấu trúc khác nhau, nhờ vào việc lựa chọn ngẫu nhiên các mẫu dữ liệu và các đặc trưng.
- **Bagging:** Random Forest sử dụng kỹ thuật bagging (bootstrap aggregating) để tạo ra các tập dữ liệu huấn luyện cho từng cây. Bagging bao gồm việc lấy mẫu ngẫu nhiên từ tập dữ liệu gốc với việc thay thế (có thể chọn một mẫu nhiều lần), sau đó huấn luyện một cây quyết định trên mỗi tập con.
- **Ngẫu nhiên:** Yếu tố "ngẫu nhiên" trong Random Forest xuất hiện ở hai khía cạnh:
 - Lựa chọn ngẫu nhiên các mẫu dữ liệu để huấn luyện mỗi cây.
 - Lựa chọn ngẫu nhiên một số thuộc tính tại mỗi nút chia nhánh trong cây.
- **Kết hợp dự đoán:**

Sau khi huấn luyện xong, mỗi cây trong rừng sẽ đưa ra một dự đoán. Trong bài toán phân loại (classification), kết quả cuối cùng được xác định bằng cách bỏ phiếu đa số (majority voting), tức là chọn lớp có nhiều cây quyết

định dự đoán nhất. Trong bài toán hồi quy (regression), kết quả cuối cùng là trung bình cộng của các dự đoán từ các cây.

Ưu điểm của Random Forest:

- **Độ chính xác cao:** Nhờ sự kết hợp của nhiều cây quyết định, Random Forest thường cho kết quả dự đoán chính xác hơn so với một cây quyết định đơn lẻ.
- **Khả năng chống overfitting tốt:** Việc sử dụng bagging và lựa chọn ngẫu nhiên các thuộc tính giúp giảm thiểu nguy cơ overfitting.
- **Hoạt động tốt trên nhiều loại dữ liệu:** Random Forest có thể áp dụng cho cả bài toán phân loại và hồi quy.

- **Ít cần điều chỉnh tham số:**

So với một số thuật toán khác, Random Forest thường không yêu cầu điều chỉnh quá nhiều tham số để đạt được hiệu suất tốt.

Ứng dụng:

Random Forest được ứng dụng rộng rãi trong nhiều lĩnh vực, bao gồm:

- Phân loại hình ảnh và văn bản
- Dự đoán giá trị
- Phân tích rủi ro
- Phát hiện gian lận
- Phân tích thị trường
- Và nhiều ứng dụng khác.

XGBoost

Today by far the most commonly used way or implementation of decision tree ensembles or decision trees there's an algorithm called XGBoost.

Boosted trees intuition

Given training set of size m

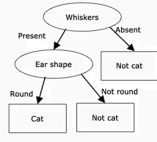
For $b = 1$ to B :

Use sampling with replacement to create a new training set of size m

But instead of picking from all examples with equal ($1/m$) probability, make it more likely to pick misclassified examples from previously trained trees

Train a decision tree on the new dataset

Ear shape	Face shape	Whiskers	Cat
Pointy	Round	Present	Yes
Floppy	Round	Absent	No
Floppy	Round	Absent	No
Pointy	Round	Present	Yes
Pointy	Not Round	Present	Yes
Floppy	Round	Absent	No
Floppy	Round	Present	Yes
Pointy	Not Round	Absent	No
Pointy	Not Round	Present	Yes



When sampling, instead of picking from all m examples of equal probability with $1/m$ probability, let's make it more likely that we'll pick misclassified examples that the previously trained trees do poorly on. In training and education, there's an idea called deliberate practice.

For example, if you're learning to play the piano and you're trying to master a piece on the piano rather than practicing the entire say five minute piece over and over, which is quite time consuming. If you instead play the piece and then focus your attention on just the parts of the piece that you aren't yet playing that well in practice those smaller parts over and over.

Then that turns out to be a more efficient way for you to learn to play the piano well. And so this idea of boosting is similar. We're going to look at the decision trees, we've trained so far and look at what we're still not yet doing well on. And then when building the next decision tree, we're going to focus more attention on the examples that we're not yet doing well.

So rather than looking at all the training examples, we focus more attention on the subset of examples is not yet doing well on and get the new decision tree, the next decision tree reporting ensemble to try to do well on them. And this is the idea behind boosting and it turns out to help the learning algorithm learn to do better more quickly.

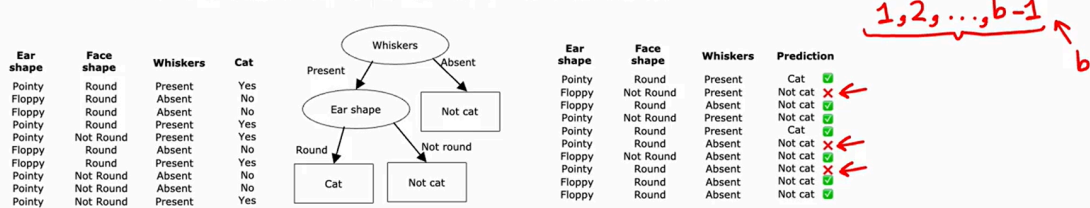
Boosted trees intuition

Given training set of size m

For $b = 1$ to B :

Use sampling with replacement to create a new training set of size m
 But instead of picking from all examples with equal $(1/m)$ probability, make it more likely to pick misclassified examples from previously trained trees

Train a decision tree on the new dataset



The green check box and the red X means if the algorithm predict correct or incorrect?

XGBoost (eXtreme Gradient Boosting)

- Open source implementation of boosted trees
- Fast efficient implementation
- Good choice of default splitting criteria and criteria for when to stop splitting
- Built in regularization to prevent overfitting
- Highly competitive algorithm for machine learning competitions (eg: Kaggle competitions)

you look at what the ensemble of trees for trees 1, 2 up through $(b-1)$, are not yet doing that well on. And when you're building tree number b , you will then have a higher probability of picking examples that the ensemble of the previously sample trees is still not yet doing well on

Using XGBoost

Classification

```
→ from xgboost import XGBClassifier  
→ model = XGBClassifier()  
→ model.fit(X_train, y_train)  
→ y_pred = model.predict(X_test)
```

Regression

```
from xgboost import XGBRegressor  
model = XGBRegressor()  
model.fit(X_train, y_train)  
y_pred = model.predict(X_test)
```

When to use decision trees

Decision Trees vs Neural Networks

Decision Trees and Tree ensembles

- Works well on tabular (structured) data
- Not recommended for unstructured data (images, audio, text)
- Fast
- Small decision trees may be human interpretable

Neural Networks

- Works well on all types of data, including tabular (structured) and unstructured data
- May be slower than a decision tree
- Works with transfer learning
- When building a system of multiple models working together, it might be easier to string together multiple neural networks

Decision trees and tree ensembles will often work well on tabular data, also called structured data. What that means is if your dataset looks like a giant spreadsheet then decision trees would be worth considering.

For example, in the housing price prediction application we had a dataset with features corresponding to the size of the house, the number of bedrooms, the number of floors, and the age at home. That type of data stored in a spreadsheet with either categorical or continuous valued features and both for classification or for regression task where you're trying to predict a discrete

category or predict a number. All of these problems are ones that decision trees can do well on.

In contrast, I will not recommend using decision trees and tree ensembles on unstructured data. That's data such as images, video, audio, and texts that you're less likely to store in a spreadsheet format. Neural networks as we'll see in a second will tend to work better for unstructured data task.

One huge advantage of decision trees and tree ensembles is that they can be very fast to train. You might remember this diagram from the previous week in which we talked about the iterative loop of machine learning development.

If your model takes many hours to train then that limits how quickly you can go through this loop and improve the performance of your algorithm. But because decision trees, including tree ensembles tend to be pretty fast to train, that allows you to go to this loop more quickly and maybe more efficiently improve the performance of your learning algorithm.

Finally, small decision trees maybe human interpretable. If you are training just a single decision tree and that decision tree has only say a few dozen nodes you may be able to print out a decision tree to understand exactly how it's making decisions. I think that the interpretability of decision trees is sometimes a bit overstated because when you build an ensemble of 100 trees and if each of those trees has hundreds of nodes, then looking at that ensemble to figure out what it's doing does become difficult and may need some separate visualization techniques.

But if you have a small decision tree you can actually look at it and see, oh, it's classifying whether something is a cat by looking at certain features in certain ways. If you've decided to use a decision tree or tree ensemble, I would probably use XGBoost for most of the applications I will work on. One slight downside of a tree ensemble is that it is a bit more expensive than a single decision tree. If you had a very constrained computational budget you might use a single decision tree but other than that setting I would almost always use a tree ensemble and use XGBoost in particular.

How about neural networks? In contrast to decision trees and tree ensembles, it works well on all types of data, including tabular or structured data as well as unstructured data. As well as mixed data that includes both structured and unstructured components. Whereas on tabular structured data, neural networks and decision trees are often both competitive on unstructured data, such as

images, video, audio, and text, a neural network will really be the preferred algorithm and not the decision tree or a tree ensemble.

On the downside though, neural networks may be slower than a decision tree. A large neural network can just take a long time to train. Other benefits of neural networks includes that it works with transfer learning and this is really important because for many applications we have only a small dataset being able to use transfer learning and carry out pre-training on a much larger dataset that is critical to getting competitive performance.

Finally, if you're building a system of multiple machine learning models working together, it might be easier to string together and train multiple neural networks than multiple decision trees. The reasons for this are quite technical and you don't need to worry about it for the purpose of this course. But it relates to that even when you string together multiple neural networks you can train them all together using gradient descent. Whereas for decision trees you can only train one decision tree at a time.